

# ExpenseEase

## Complete Mid-Semester Master Plan

Product Roadmap, SDLC Model, Requirements, Design, Testing, Project Management,  
Demo, and Submission Plan

Prepared for Software Engineering Mid-Semester Evaluation

Group: <<FILL: group number>> | Members: <<FILL: names and registration numbers>>

# Contents

1. Product Roadmap and Official Evaluation Checklist
2. SDLC Model and Mid-Semester Increment Plan
3. Mid-Semester Project Report Source
4. Software Requirements Specification
5. Use Cases and User Stories
6. Architecture Design
7. Database Design
8. UML Diagrams
9. UI Wireframes
10. Testing Plan and Current Evidence
11. Project Plan
12. Task Allocation
13. Weekly Progress Template
14. Meeting Log Template
15. Contribution Record Template
16. Risk Register
17. Demonstration Script
18. Submission Checklist

# 1. Product Roadmap and Official Evaluation Checklist

## ExpenseEase Feature Roadmap

### 1. Product vision

ExpenseEase is a shared-expense management system for housemates, trips, friends, families, couples, and event groups. Its purpose is to reduce awkward conversations and manual calculations by recording expenses, calculating balances, showing who owes whom, and helping members settle fairly.

ExpenseEase is inspired by the shared-expense problem space. It must use its own branding, interface, code, wording, and design; it must not copy Splitwise source code, assets, or protected material.

### 2. Mid-semester MVP

The mid-semester evaluation does not require a complete commercial product. The required target is approximately 40-60% progress and at least one working end-to-end feature.

The selected SDLC approach is Incremental-Iterative Agile. The increment sequence, lifecycle evidence, risk reviews, testing plan, reuse plan, and maintenance plan are documented in 06\_Project\_Management/SDLC\_Model\_and\_MidSem\_Plan.md.

#### Must demonstrate

Feature | Status / action

Register, login, logout | Backend complete; add registration UI

Create group and view groups | Backend complete; add create-group UI

Add registered group member | Backend complete; add member UI

Add expense | Backend complete; equal-split UI complete

Equal, exact, and percentage splits | Backend complete; exact/percent UI remains

View group balances | Complete

Simplify debts | Complete

Record settlement | Backend complete; add settle-up UI

Activity feed | Complete

Dashboard summary | Backend complete; add UI

Automated tests | Initial tests complete; expand evidence

Requirements, designs, report, plan | Drafts complete; fill team evidence and export PDFs

#### Minimum demonstration flow

1. Log in with a seeded account.
2. Open a shared group.
3. Add an equal-split expense.
4. Open the balance tab.
5. Show debt-simplification suggestions.

6. Open the activity feed.

## 3. Core product features

### 3.1 Users and security

- Register, log in, log out, reset password, and change password.
- JWT or secure cookie session management.
- Email verification and optional social sign-in.
- Profile: name, avatar, preferred currency, language, time zone.
- Account deletion and data export.
- Role-based access: group admin, member, viewer.
- Audit log for sensitive changes.

### 3.2 Groups and people

- Create groups for homes, trips, couples, events, friends, and family.
- Add/remove members and invite by email or shareable invite link.
- Group description, image, default currency, category rules, and budgets.
- Member search and contact list.
- Group archive and restore.
- Multiple subgroups for large trips or households.

### 3.3 Expense capture

- Add expense with description, amount, date, category, note, payer, and participants.
- Multiple payers for a single bill.
- Equal split.
- Exact-amount split.
- Percentage split.
- Share/ratio split.
- Itemized split: assign receipt items to participants.
- Tax, tip, discount, service charge, and delivery-fee allocation.
- Expense attachments and receipt images.
- Edit, delete, duplicate, and comment on expenses.
- Expense search, filters, sorting, tags, and export.
- Bulk import from CSV/bank statement.

### 3.4 Balances and settlements

- Per-group and global balances.
- “You owe”, “you are owed”, and net balance cards.
- Deterministic debt simplification.

- Settlement suggestions with minimal practical transactions.
- Record cash, bank, UPI, card, or wallet settlement.
- Partial settlement and settlement history.
- Payment links or payment-provider integration in a later phase.
- Dispute/confirm a settlement workflow.

### 3.5 Recurring and scheduled expenses

- Weekly, monthly, quarterly, and yearly recurring expenses.
- Start/end dates and reminders.
- Typical use cases: rent, electricity, internet, subscriptions, groceries.
- Pause, resume, skip, and edit one occurrence or all future occurrences.
- Scheduled job that creates due expenses.

### 3.6 Reports and analytics

- Spending by category, member, group, and date range.
- Monthly trend charts.
- Budget versus actual spending.
- Export CSV/PDF statements.
- Group settlement history.
- Top categories and unusual-spend indicators.

### 3.7 Multi-currency and international trips

- Store original amount and group base currency.
- Currency conversion at the expense date.
- Show exchange-rate source and timestamp.
- Manual override for offline or agreed group rates.
- Avoid live exchange-rate dependencies in the mid-semester MVP.

### 3.8 Notifications and collaboration

- In-app notifications for invitations, new expenses, edits, and settlements.
- Email or push notifications in later releases.
- Due-balance reminders.
- Comments and reactions on expenses.
- Optional digest notifications.

### 3.9 Platform and usability

- Responsive web application.
- Progressive web app support and offline draft capture.
- Native Android/iOS apps in a later release.

- English first, followed by localization support.
- Accessibility: labels, keyboard navigation, contrast, screen-reader-friendly controls.

## 4. Suggested technical architecture for future features

Capability | Suggested implementation

Core API | React, Node.js, Express, Prisma, SQLite during development

Production database | PostgreSQL

Background tasks | Queue worker for recurring bills, imports, reminders

File storage | Secure object storage for receipt attachments

Search | Database full-text search initially

Analytics | Aggregated read models and chart API

Observability | Structured logs, error tracking, metrics

## 5. Data-model additions needed

Feature | New or changed records

Invitations | GroupInvite

Multiple payers | ExpensePayment

Itemized receipt | Receipt, ReceiptItem, ExpenseItemSplit

Recurring expenses | RecurringExpense, RecurrenceRun

Attachments | Attachment

Comments | ExpenseComment

Notifications | Notification, NotificationPreference

Currency | CurrencyRate, originalAmount, originalCurrency

Budgeting | GroupBudget, CategoryBudget

Auditability | AuditLog

## 6. Delivery roadmap

### Release 0.1 - Mid-semester MVP

- Current secure backend.
- Groups, expenses, splits, balances, simplification, settlements.
- One working frontend end-to-end flow.
- Tests, diagrams, report, demo video, and package.

### Release 0.2 - Semester completion

- Complete all expense and settlement frontend workflows.
- Recurring expenses.
- Better activity, filters, search, and exports.

- Dashboard charts and category analytics.
- More test coverage and deployment.

## Release 1.0 - Production readiness

- PostgreSQL, secure deployment, backup/recovery.
- Stronger access controls and audit log.
- Performance testing and monitoring.
- Privacy policy, data retention, and consent controls.
- Optional payment and notification integrations.

## 7. Mid-semester checklist

- App runs from README instructions.
- Seeded demo data works.
- One complete end-to-end feature is demonstrated.
- Lint, tests, and build pass.
- At least 35 truthful test cases and evidence are included.
- Report includes real screenshots.
- All team details and contribution claims are completed honestly.
- PDF files are exported and visually checked.
- Demo video is recorded and accessible.
- Final ZIP follows the required naming and folder layout.

## 8. Official mid-semester evaluation deliverables

This section translates the course evaluation PDF into concrete project work. These are submission requirements, not optional product features.

### 8.1 Expected project progress

- Clearly defined problem statement and measurable objectives.
- Stakeholders, included scope, excluded scope, assumptions, and constraints.
- Functional and non-functional requirements with priorities.
- Use-case diagram/descriptions or user stories with acceptance criteria.
- Architecture and database/data-model design.
- UI wireframes or working prototype.
- At least one real end-to-end feature.
- Unit or functional tests of implemented features.
- Project plan and confirmed contribution record for every member.

### 8.2 Mandatory report content

Create one PDF report, recommended length 10-15 pages excluding appendices, containing:

1. Cover page: title, group number, all member names/registration numbers, course/section, instructor.
2. Problem statement and motivation.
3. Project objectives.
4. Scope: included features, excluded features, assumptions, constraints.
5. Stakeholder identification.
6. Functional requirements, non-functional requirements, and prioritization.
7. Requirements model: use cases and/or user stories with acceptance criteria.
8. System design: architecture, modules, data model/ER diagram, relevant UML.
9. Technology stack and justification.
  10. Implementation progress, completed/in-progress work, real screenshots, key decisions.
  11. Testing: test cases, input, expected result, actual result, pass/fail, real evidence.
  12. Project-management status: plan, progress, pending work, risks, mitigations.
  13. Individual contribution table.
  14. Genuine challenges and attempted solutions.
  15. Remaining-semester plan.
  16. Real references.

### 8.3 Mandatory source-code package

- Meaningful README with setup, run, test, build, and database instructions.
- Required dependencies and sample configuration file.
- Sample input or seed data.
- Clear folder organization and only necessary comments.
- No secrets, passwords, API keys, real databases, or confidential data.
- Verify a fresh clone runs by following the README exactly.

### 8.4 Mandatory requirements and design artifacts

- SRS/requirements document.
- Use-case diagram/user-story document.
- Architecture diagram.
- Database design/data model.
- Relevant UML diagrams.
- UI wireframes/prototype.
- Test-case document.
- Keep editable Mermaid/Markdown sources alongside PDF exports where practical.

### 8.5 Mandatory demo video

Record a clear 5-7 minute video covering:

1. Project title and team introduction.
2. Problem explanation.

3. Proposed solution.
4. Architecture/design explanation.
5. Working-feature demonstration.
6. Current limitations.
7. Remaining work.
8. Each member's genuine contribution.

If hosted externally, use an institutionally permitted service, set the link to view-only, verify access without sign-in, and place it in the Moodle comment, report, and DEMO\_VIDEO\_LINK.txt.

## 8.6 Mandatory project-management evidence

- Task-allocation table.
- Weekly progress record.
- Meeting log.
- Individual contribution declaration signed or confirmed by each member.

## 8.7 Final ZIP structure and naming

Use the name:

Section\_GroupNumber\_ExpenseEase\_MidSem.zip

Required folders:

01\_Report/MidSem\_Project\_Report.pdf  
02\_Requirements/SRS.pdf  
02\_Requirements/Use\_Cases\_or\_User\_Stories.pdf  
03\_Design/Architecture\_Diagram.pdf  
03\_Design/Database\_Design.pdf  
03\_Design/UML\_Diagrams.pdf  
04\_Source\_Code/README.md  
04\_Source\_Code/source\_files/  
04\_Source\_Code/sample\_data/  
05\_Testing/Test\_Cases.pdf  
05\_Testing/Test\_Evidence/  
06\_Project\_Management/Project\_Plan.pdf  
06\_Project\_Management/Meeting\_Log.pdf  
06\_Project\_Management/Contribution\_Record.pdf  
07\_Demo/DEMO\_VIDEO\_LINK.txt

## 9. Important scope recommendation

Do not attempt every feature, native app, payment integration, and cloud deployment for the mid-semester evaluation. Present later work as a documented roadmap. For the evaluation, prioritize a stable, explainable, tested shared-expense

MVP with one strong end-to-end demonstration.

## 2. SDLC Model and Mid-Semester Increment Plan

### ExpenseEase SDLC Model and Mid-Semester Plan

#### 1. Selected software-development model

ExpenseEase uses an **\*\*Incremental-Iterative Agile development model\*\***.

The overall system is divided into prioritized, usable increments. Each increment follows a small lifecycle:

Requirements -> Design -> Implement -> Test -> Review -> Next increment

This selection is aligned with the class material on incremental, iterative, RAD, Agile, requirements engineering, software design, testing, risk management, reuse, estimation, and maintenance.

#### 2. Why this model fits ExpenseEase

Project condition | Model response

Shared-expense features can be divided into independent functional slices | Incremental delivery provides a working feature early

UI requirements need refinement after a prototype is visible | Iterative reviews allow controlled improvement

Mid-semester evaluation requires working progress, not the complete final product | The first increments provide demonstrable evidence

Requirements such as split types and settlement UI can evolve | Agile backlog prioritization manages change

Correct money calculations and authorization are high-risk | Every increment includes unit/API tests and review

The team has a limited academic-semester timeline | Small releases keep work measurable and explainable

The project is **\*\*not presented as a pure Waterfall model\*\***, because the full UI and later features are still evolving. It is **\*\*not presented as a pure Spiral model\*\***, because the project does not have the scale, cost, or formal risk-prototyping cycles required to justify a full Spiral process. Risk management is still applied within each increment.

#### 3. Relation to models taught in class

Taught topic | How ExpenseEase applies it

Incremental model | Deliver working parts: auth/groups, expenses/splits, balances/settlements, frontend, then completion

Iterative model | Improve UI, validation, documentation, and tests after each working slice

Agile development | Use a prioritized backlog, short implementation cycles, review, and adaptation

RAD/prototyping | Use early wireframes and a running frontend prototype to validate the user flow

Requirements engineering | Maintain SRS, user stories, use cases, priorities, traceability, and acceptance criteria

Software design | Use layered routes -> controllers -> services -> Prisma architecture and modular components

Testing | Plan, design, execute, evaluate, and record evidence for unit, API, frontend, and manual tests

Risk management | Review rounding, authorization, data loss, integration, deadline, and demo risks

Estimation | Use a work-breakdown structure and revise planned work as evidence changes

Software reuse | Reuse established frameworks/libraries only after checking compatibility, licensing, security, and tests

Maintenance/evolution | Plan corrective, adaptive, perfective, and preventive changes after the core release

## 4. Increment plan

### Increment 0 - Planning and requirements

**Goal:** establish a feasible scope and reviewable project foundation.

- Problem statement, objectives, stakeholders, and scope.
- Functional and non-functional requirements.
- User stories, use cases, wireframes, architecture, ER design, UML.
- Product backlog, risks, task allocation, and initial plan.

**Evidence:** SRS, design documents, project plan, risk register, backlog, and meeting records completed with real team details.

### Increment 1 - Secure foundation and group management

**Goal:** users can authenticate and establish a shared group.

- Register, login, logout, and authenticated-user endpoint.
- Password hashing, JWT, validation, centralized errors, rate limiting.
- Create/list/view groups.
- Admin creation and group-member authorization.
- Add registered group members.

**Current status:** backend implemented and tested. Registration/create-group/member UI remains to be completed.

### Increment 2 - Expense recording and split calculation

**Goal:** a group member can record an expense with correct financial arithmetic.

- Expense CRUD.
- Equal, exact, and percent split calculation.
- Integer-paise storage.
- Deterministic remainder distribution.
- Payer/participant membership validation.
- Expense activity log.

**Current status:** backend implementation and service tests complete. Equal-split UI is complete; exact/percent UI and edit/delete UI remain.

### Increment 3 - Balance, simplification, and settlement

**Goal:** members understand balances and can reduce debt complexity.

- Compute balances from expenses and settlements.
- Assert the sum-zero balance invariant.
- Debt-simplification service.
- Record settlement.

- Activity feed and dashboard totals.

**\*\*Current status:\*\*** backend implementation complete; balances/activity UI complete. Settlement and dashboard UI remain.

## Increment 4 - Integrated frontend and usability

**\*\*Goal:\*\*** provide an accessible, demonstrable end-to-end user experience.

- Register and login pages.
- Protected routes and group dashboard.
- Create group/add member screens.
- Add/Edit expense modal with all split types.
- Balances, settle-up, activity, dashboard.
- Loading, empty, error, responsive, and accessibility states.

**\*\*Current status:\*\*** in progress. Login, group browsing, equal expense, balance, and activity flow work.

## Increment 5 - Quality, evidence, and release package

**\*\*Goal:\*\*** prepare verified mid-semester submission evidence.

- Expand unit, API, and frontend tests.
- Record real test output, coverage, lint output, and screenshots.
- Complete report, project-management evidence, and demo script.
- Export verified PDFs and build final ZIP.
- Record the demonstration video.

**\*\*Current status:\*\*** in progress.

## 5. Iteration cycle used for every increment

Step | Team activity | Required evidence

Plan | Select prioritized backlog items and define acceptance criteria | Task allocation/update

Analyse | Confirm requirements, constraints, users, and risks | SRS/user-story update

Design | Update module interface, data model, wireframe, or diagram | Design artifact

Implement | Build a small cohesive vertical slice | Source code and reviewed change record

Test | Run relevant unit, API, frontend, and manual checks | Real test result/evidence

Review | Demonstrate the slice and identify improvements | Meeting/progress record

Adapt | Reprioritize remaining work and update risks | Updated plan/backlog

## 6. Mid-semester work breakdown structure

WBS | Work package | Output | Status

1.0 | Requirements | SRS, priorities, user stories/use cases | Draft complete; review/fill required

2.0 | Design | Architecture, ER, UML, wireframes, OpenAPI | Draft complete; review/fill required

3.0 | Backend foundation | Auth, groups, security, database | Implemented  
4.0 | Expense domain | Expenses, splits, activity | Implemented  
5.0 | Financial domain | Balances, simplification, settlements, dashboard | Backend implemented  
6.0 | Frontend | Full screens and modals | Partially implemented  
7.0 | Testing | Automated and manual test evidence | Initial tests complete; expansion required  
8.0 | Documentation | Report, management, demo, PDFs | Drafts complete; evidence/PDF export required  
9.0 | Packaging | ZIP build, clean-clone check, submission | Pending

## 7. Risk-driven review within the Agile increments

This is not a full Spiral model, but the team performs a small risk review before and after each increment.

Risk | Impact | Increment response

Incorrect split rounding | Incorrect balances | Integer pause and deterministic calculator tests

Unauthorized group access | Privacy/data-integrity failure | JWT and membership/admin service checks

Balances become stale after edits | Incorrect debt suggestions | Compute balances from records, do not store them

UI/backend integration failure | Demo failure | Test API contracts and rehearse seeded demo

Scope creep | Incomplete mid-semester package | Keep advanced features in future backlog

Database setup failure | App cannot run locally | README, env example, seed, migration evidence

Weak contribution evidence | Marks lost | Maintain task allocation, meeting, and progress records

## 8. Testing plan mapped to lifecycle

Lifecycle stage | Testing activity

Requirements | Define acceptance criteria and traceability to test cases

Design | Review data model, interfaces, authorization boundaries, and testability

Implementation | Unit-test split calculation, simplification, and balances

Integration | Test API routes with isolated SQLite database

System/demo | Test seeded login-to-expense-to-balance flow in browser

Release | Run lint, test, coverage, build, seed, secrets scan, and ZIP inspection

## 9. Reuse and maintenance plan

### Reuse

ExpenseEase reuses standard, well-supported components: React, Express, Prisma, SQLite, Zod, bcrypt, JWT tooling, Vitest, and Supertest. Reuse does not remove the need for design, licensing review, compatibility checks, security review, integration tests, or documentation.

### Maintenance after the semester release

- **Corrective:** fix defects such as rounding, validation, or UI bugs.
- **Adaptive:** update dependencies or database configuration when environments change.
- **Perfective:** add reports, recurring bills, search, analytics, and UI improvements.

- **Preventive:** refactor duplicated code, improve tests, update documentation, and address technical debt.

## 10. Mid-semester review statement

For the evaluation, the team will demonstrate Increment 1 through the working parts of Increment 4, supported by requirements/design artifacts, automated tests, a real end-to-end flow, project-management records, and a transparent remaining-work plan. The team must replace all fill-in markers and progress claims with verified team evidence before submission.

## 3. Mid-Semester Project Report Source

### ExpenseEase - Mid-Semester Project Report

#### 1. Cover page

**\*\*Project:\*\*** ExpenseEase

**\*\*Group number:\*\*** <<FILL: group number>>

**\*\*Members and registration numbers:\*\*** <<FILL: members>>

**\*\*Course and section:\*\*** <<FILL: course and section>>

**\*\*Supervisor/instructor:\*\*** <<FILL: instructor>>

#### 2. Problem statement and motivation

Friends, roommates, and trip groups often use chat messages or spreadsheets to track shared spending. This causes forgotten IOUs, unclear payers, unequal splits, and circular debts. ExpenseEase provides an auditable local web application to record group expenses, compute exact balances, and recommend a smaller set of repayments.

#### 3. Project objectives

1. Record expenses with equal, exact, and percentage split rules.
2. Preserve monetary accuracy using integer paise.
3. Compute net balances and simplified debt transfers.
4. Protect group data with authentication and authorization.
5. Maintain an explainable, tested codebase.

#### 4. Scope

Included scope is authentication, group membership, expenses, balances, settlements, activity, and dashboard summaries. Current implementation includes backend endpoints and a frontend demo flow for login, group browsing, equal expense entry, balances, and activity. Payment processing, OCR, multi-currency, social login, notifications, native apps, and recurring expenses are excluded. The application uses INR, English UI, local SQLite, and manually recorded settlements.

#### 5. Stakeholders

Stakeholder | Interest | Influence

Group member | Accurate shared costs | High

Group admin | Membership and data oversight | High

Course evaluator | Evidence of engineering practice | High

Development team | Deliverable quality and learning | High

Future maintainer | Clear design and tests | Medium

#### 6. Requirements specification

Functional requirements cover registration, group creation, member addition, expense CRUD, split validation, balance calculation, debt simplification, settlement recording, activity, and dashboard summaries. Non-functional requirements include bcrypt password hashing, JWT expiry, validated input, SQLite portability, accessible labels, and balance-sum-zero invariants.

## 7. Requirements model

The detailed use-case diagram, six use cases, and eighteen user stories are in the requirements document. Core acceptance requirement: submitted expense shares must total exactly to the stored paise amount, and only group members can access a group.

## 8. System design

The browser SPA calls an Express REST API. Routes delegate to controllers and services; Prisma accesses SQLite. The debt simplifier is a pure service, independent of database access. Balances are computed from expenses and settlements rather than stored, preventing stale records.

## 9. Technology stack and justification

Layer | Choice | Why

Frontend | React/Vite/Axios | Fast local SPA and API integration

Backend | Node.js/Express | Small, familiar REST service

Validation | Zod | Declarative input contracts

Database | SQLite/Prisma | Zero-setup relational model

Security | bcrypt/JWT/helmet | Password hashing and protected routes

Tests | Vitest/Supertest | Fast unit and API-oriented testing

## 10. Implementation progress

Completed backend capabilities: auth, groups, member authorization, expense CRUD, deterministic split calculators, balance calculation, simplification, settlement recording, activity, dashboard, seed data, and unit/service tests. Current frontend implements a live login-to-group-to-equal-expense-to-balance workflow. Exact/percentage split UI, settlement UI, broader test coverage, and polished responsive screens remain in progress.

Screenshots to capture from a real run: login\_demo.png, groups\_dashboard.png, group\_expenses\_empty.png, add\_equal\_expense.png, expenses\_after\_add.png, balances\_suggestions.png, activity\_feed.png, and test\_results.png.

## 11. Testing performed

The current real automated run includes 13 backend tests and 1 frontend test, with lint and production build passing. Covered evidence includes equal remainder handling, invalid exact splits, percentage rounding, deterministic debt suggestions, balance invariants, health response, group authorization, expense edit/delete authorization, and persisted balance display. No unexecuted scenario is marked as passed.

## 12. Project-management status

The Gantt plan and risk register are in the project-management documents. Backend core services are ahead of frontend completeness; remaining risk is documentation, evidence, full UI coverage, packaging, and team-provided contribution

records.

### 13. Individual contribution table

Member | Tasks done | Modules / commits | Percentage

<<FILL: name>> | <<FILL>> | <<FILL>> | <<FILL>>

### 14. Challenges faced and solutions attempted

The project addressed deterministic rounding by allocating leftover paise in ascending user-id order. Simplification matches largest creditor and debtor deterministically. Authorization is enforced through membership and creator/admin checks. Balance consistency is maintained by calculating from records, not caching balances. <<CONFIRM: add only genuine team challenges>>

### 15. Remaining-semester work

Complete exact/percentage expense UI and settlement UI, expand API and frontend tests, add screenshots and test evidence, generate migration evidence, improve accessibility and responsiveness, conduct usability testing, and prepare the verified submission package.

### 16. References

1. Prisma Documentation, <https://www.prisma.io/docs>
2. Express Documentation, <https://expressjs.com/>
3. React Documentation, <https://react.dev/>
4. OWASP Top 10, <https://owasp.org/www-project-top-ten/>
5. Ian Sommerville, Software Engineering.

## 4. Software Requirements Specification

### ExpenseEase Software Requirements Specification

#### Purpose and scope

ExpenseEase records shared group expenses, calculates balances, and proposes simplified settlement transfers. It supports local, single-currency INR use. Payment processing, OCR, mobile apps, notifications, recurring expenses, social login, and multi-currency support are out of scope.

#### Functional requirements

ID | Requirement | Priority

FR-01 | Users register and authenticate securely. | Must

FR-02 | Members create and manage groups. | Must

FR-03 | Members add equal, exact, or percentage expenses. | Must

FR-04 | The system computes balances from immutable records. | Must

FR-05 | The system suggests deterministic simplified debts. | Must

FR-06 | Authorized users record settlements. | Should

#### Non-functional requirements

Passwords use bcrypt with cost 10 or greater. Money is stored as integer paise and balance sums must equal zero. Inputs are validated before persistence. The local SQLite implementation targets balance calculation for 1,000 expenses in under 500 ms.

#### Traceability

FR | User story | Test

FR-03 | As a member, I want to split an expense fairly. | services.test.js

FR-04 | As a member, I want accurate balances. | services.test.js

FR-05 | As a member, I want fewer transfers. | services.test.js

## 5. Use Cases and User Stories

### Use Cases and User Stories

```
flowchart LR
  Member((Member)) --> Register
  Member --> CreateGroup[Create group]
  Member --> AddExpense[Add expense]
  Member --> ViewBalances[View balances]
  Member --> Simplify[View simplified debts]
  Member --> Settle[Record settlement]
```

ID | Use case | Preconditions | Main outcome

UC-01 | Register / login | Visitor has valid details | Authenticated session is issued

UC-02 | Create group | User is authenticated | Creator becomes group admin

UC-03 | Add expense | User is a group member | Valid splits total the amount

UC-04 | View balances | User is a group member | Computed net balances are shown

UC-05 | Simplify debts | Balances sum to zero | Deterministic suggested transfers are shown

UC-06 | Record settlement | Both users are members | Settlement affects computed balances

### User stories

1. As a visitor, I want to register so that I can use the system.
2. As a member, I want to log in so that my groups are protected.
3. As a member, I want to create a group so that I can share costs.
4. As an admin, I want to add registered people so that they can participate.
5. As a member, I want to view my groups so that I can choose one quickly.
6. As a member, I want to add an equal split so that everyone pays fairly.
7. As a member, I want to add exact shares so that custom amounts are supported.
8. As a member, I want to add percentage shares so that ratios are supported.
9. As a member, I want paise rounding to be deterministic so that totals are exact.
  10. As a member, I want to see expenses so that I can audit the group.
  11. As a creator, I want to edit my expense so that mistakes can be corrected.
  12. As an admin, I want to edit an expense so that the group data stays accurate.
  13. As a member, I want to delete an erroneous expense.
  14. As a member, I want net balances so that I know whether I owe money.
  15. As a member, I want simplified debts so that fewer payments are needed.
  16. As a member, I want to record a settlement so that balances update.
  17. As a member, I want an activity feed so that group changes are visible.
  18. As a member, I want dashboard totals so that I understand my overall position.

Acceptance criterion shared by expense stories: Given I am a group member, when I submit valid data, then the split shares sum exactly to the stored amount and the activity is recorded.

## 6. Architecture Design

### Architecture

```
flowchart LR
  Browser[React / Vite SPA] --> API[Express REST API]
  API --> Services[Business services]
  Services --> Prisma[Prisma ORM]
  Prisma --> SQLite[(SQLite)]
```

Routes handle transport, controllers coordinate requests, and services own business calculations. The debt simplifier is deliberately pure and has no database access.

## 7. Database Design

### Database Design

```
erDiagram
    USER ||--o{ GROUP_MEMBER : joins
    GROUP ||--o{ GROUP_MEMBER : has
    GROUP ||--o{ EXPENSE : contains
    EXPENSE ||--o{ EXPENSE_SPLIT : divides
    GROUP ||--o{ SETTLEMENT : records
```

Amounts are integer paise. ExpenseSplit normalizes per-person liability, while balances are calculated rather than stored.

## 8. UML Diagrams

### UML Diagrams

#### Add expense sequence

```
sequenceDiagram
participant U as Member
participant A as API
participant S as Expense service
participant D as SQLite
U->>A: POST expense
A->>S: validate and calculate splits
S->>D: store expense, splits, activity
D-->>A: expense
A-->>U: 201 Created
```

#### Balance activity

```
flowchart TD
A[Load members, expenses, settlements] --> B[Compute net paise]
B --> C{Sum equals zero?}
C -->|yes| D[Sort creditors and debtors]
D --> E[Match largest amounts]
E --> F[Return transfers]
C -->|no| G[Raise invariant error]
```

#### Backend class view

```
classDiagram
class GroupService
class ExpenseService
class BalanceService
class DebtSimplifier
GroupService --> PrismaClient
ExpenseService --> PrismaClient
BalanceService --> DebtSimplifier
```

# 9. UI Wireframes

## Wireframes

```
+-----+
| ExpenseEase |
| Email [ ] |
| Password [ ] |
| [ Log in ] |
+-----+

+-----+
| Goa Weekend |
| Expenses | Balances | Feed|
| Add equal-split expense |
| Description [ ] Amount [ ]|
| Payer [v] [Add expense] |
+-----+
```

Login is keyboard-accessible and fields have labels. Group detail uses tabs for expenses, balances, and activity; the current frontend provides the equal split workflow. Exact/percent UI and settle-up modal remain planned.

## 10. Testing Plan and Current Evidence

### Test Cases

ID | Module | Description | Expected Result | Actual Result | Pass/Fail | Evidence

TC-01 | Splits | Equal 10,000 paise across three users | Shares total 10,000 and remainder is deterministic | Automated assertion passed | Pass | server/tests/services.test.js

TC-02 | Splits | Reject invalid exact split | Validation error | Automated assertion passed | Pass | server/tests/services.test.js

TC-03 | Debts | Simplify two debtors to one creditor | Two transfers | Automated assertion passed | Pass | server/tests/services.test.js

TC-04 | Balances | Compute paid minus shares | Sum is zero | Automated assertion passed | Pass | server/tests/services.test.js

TC-05 | API | Health endpoint | JSON `ok: true` | Automated assertion passed | Pass | server/tests/auth.test.js

Additional API and UI scenarios must be executed and recorded as their endpoints are implemented; no unrun scenario is labelled passed.

# 11. Project Plan

## Project Plan

```
gantt
title ExpenseEase semester plan
dateFormat YYYY-MM-DD
section Completed
Foundation and auth :done, 2026-08-01, 14d
Groups, expenses, balances :done, 2026-08-15, 21d
section Remaining
Frontend completion :2026-09-10, 14d
Testing and documentation :2026-09-24, 14d
Final packaging :2026-10-08, 7d
```

Current status: backend foundation and one frontend end-to-end flow are implemented. Exact/percentage UI, settlement UI, API integration testing, full documentation, video, and packaging remain pending. Do not replace this status with a percentage unless the team confirms it.

## 12. Task Allocation

### Task Allocation

| Task                    | Owner            | Status               | Dates    |
|-------------------------|------------------|----------------------|----------|
| Requirements and design | <<FILL: member>> | In progress          | <<FILL>> |
| Backend API             | <<FILL: member>> | In progress          | <<FILL>> |
| Frontend                | <<FILL: member>> | In progress          | <<FILL>> |
| Testing and demo        | <<FILL: member>> | Pending confirmation | <<FILL>> |

# 13. Weekly Progress Template

## Weekly Progress

| Week | Goals    | Done     | Blocked  | Next     |
|------|----------|----------|----------|----------|
| 1    | <<FILL>> | <<FILL>> | <<FILL>> | <<FILL>> |
| 2    | <<FILL>> | <<FILL>> | <<FILL>> | <<FILL>> |
| 3    | <<FILL>> | <<FILL>> | <<FILL>> | <<FILL>> |
| 4    | <<FILL>> | <<FILL>> | <<FILL>> | <<FILL>> |
| 5    | <<FILL>> | <<FILL>> | <<FILL>> | <<FILL>> |
| 6    | <<FILL>> | <<FILL>> | <<FILL>> | <<FILL>> |

# 14. Meeting Log Template

## Meeting Log

| Date     | Attendees | Agenda   | Decisions | Action items |
|----------|-----------|----------|-----------|--------------|
| <<FILL>> | <<FILL>>  | <<FILL>> | <<FILL>>  | <<FILL>>     |
| <<FILL>> | <<FILL>>  | <<FILL>> | <<FILL>>  | <<FILL>>     |
| <<FILL>> | <<FILL>>  | <<FILL>> | <<FILL>>  | <<FILL>>     |
| <<FILL>> | <<FILL>>  | <<FILL>> | <<FILL>>  | <<FILL>>     |
| <<FILL>> | <<FILL>>  | <<FILL>> | <<FILL>>  | <<FILL>>     |
| <<FILL>> | <<FILL>>  | <<FILL>> | <<FILL>>  | <<FILL>>     |

# 15. Contribution Record Template

## Contribution Record

Member | Tasks | Modules / commits | Percentage

<<FILL: name>> | <<FILL>> | <<FILL>> | <<FILL>>

<<FILL: name>> | <<FILL>> | <<FILL>> | <<FILL>>

Declaration: <<FILL: name>> confirms the record above. Signature/Confirmed: \_\_\_\_\_ Date: \_\_\_\_\_

## 16. Risk Register

### Risk Register

| ID  | Risk                  | Likelihood | Impact | Mitigation                      | Owner    |
|-----|-----------------------|------------|--------|---------------------------------|----------|
| R1  | Split rounding defect | Medium     | High   | Unit tests and paise arithmetic | <<FILL>> |
| R2  | Scope creep           | Medium     | Medium | Freeze excluded features        | <<FILL>> |
| R3  | Member unavailable    | Medium     | High   | Early task allocation           | <<FILL>> |
| R4  | Integration bug       | Medium     | High   | API tests and demo rehearsal    | <<FILL>> |
| R5  | Security flaw         | Low        | High   | JWT, bcrypt, validation         | <<FILL>> |
| R6  | Local data loss       | Low        | Medium | Seed and backups                | <<FILL>> |
| R7  | Deadline crunch       | High       | High   | Weekly checkpoints              | <<FILL>> |
| R8  | Tool/version issue    | Medium     | Medium | Lock dependencies               | <<FILL>> |
| R9  | Unclear contributions | Medium     | High   | Confirmed contribution log      | <<FILL>> |
| R10 | Demo failure          | Medium     | High   | Fresh seed and checklist        | <<FILL>> |

# 17. Demonstration Script

## Demo Script (5-7 minutes)

Time | Presenter action

0:00-0:30 | Introduce ExpenseEase and <<FILL: team members>>.

0:30-1:00 | Explain shared-expense pain points.

1:00-1:45 | Describe groups, accurate paise calculations, and simplified debts.

1:45-2:45 | Explain React, Express, Prisma, SQLite, and the service layer.

2:45-5:15 | Log in with seed user, open Goa Weekend, add an equal expense, open Balances, view suggested settlement, open Activity.

5:15-5:45 | State current limitations: exact/percent UI and settle-up UI are pending.

5:45-6:15 | State remaining work: test expansion, documentation, packaging.

6:15-7:00 | Each member states their verified contribution.

Before recording: run a fresh seed, use 110-125% browser zoom, close notifications, and confirm readable text and audio.

## 18. Submission Checklist

### Submission Checklist

- Fill group, student, course, section, and instructor details.
- Confirm all <<FILL>> markers with genuine information.
- Capture real UI and test-evidence screenshots.
- Export the report and artifact PDFs.
- Verify the video link opens without requesting access.
- Run lint, tests, build, seed, and secrets scan.
- Verify the ZIP opens and has the required name.
- One group member uploads the final verified package.